

WALL-E: RL-based planning for two-wheeled robot

Maksim Alymov
Skolkovo Institute
of science and technology
Moscow, Russia
Maksim.Alymov@skoltech.ru

Andrey Filin
Skolkovo Institute
of science and technology
Moscow, Russia
Andrey.Filin@skoltech.ru

Egor Miroshnichenko
Skolkovo Institute
of science and technology
Moscow, Russia
egor.miroshnichenko@skoltech.ru

Alexey Morozov
Skolkovo Institute
of science and technology
Moscow, Russia
Alexey.Morozov@skoltech.ru

Timur Nabiev
Skolkovo Institute
of science and technology
Moscow, Russia
Timur.Nabiev@skoltech.ru

Abstract—Many planning algorithms rely on knowledge of transition functions - information about dynamics. But on real world usually we don't have such information, either we have knowledge is incomplete. In this work we propose method of modeling robot motion via MDP([1]) and the method of solving it via Q-learning([2]) algorithm. Such approach allows us to deal with environments with uncertainties and unknown dynamics.

Index Terms—Scientific writing, Typesetting, Document creation, Syntax

I. INTRODUCTION

In a two-wheeled robot, the wheels are mounted on the sides of the robot body. There is an empty space between the wheels under the body, so the robot is able to pass obstacles of a certain size between the wheels when moving.

This kind of problem is relevant also for robots that move, for example, on the surface of the Moon/Mars, and in this case we do not have any information about the environment, so it is necessary to implement an algorithm that will not rely on predefined transition functions.

II. METHODS

A. Modeling of the environment

We used OpenAI gymnasium framework to build a custom environment to test our approach and evaluate it's efficiency.

After some tests of the complete algorithm with environments of different size, we have chosen environment of a small size (20, 20) in order to perform training faster.

The most tricky thing in the environment development is "reward design" in our case we rewarded the agent such way:

- +1, if agent desired the goal
- -1, if agent collided
- 0, other ways

The important part is that the robot must be able to pass obstacles between its' wheels, so we have parameterized our environment for each possible rotation of the robot (0, 45, 90 and 135 degrees) and convolve the objects, so when the robot need to pass the obstacle between wheels, he becomes a point and obstacles are redefined. You can see the collision maps below for clarity:

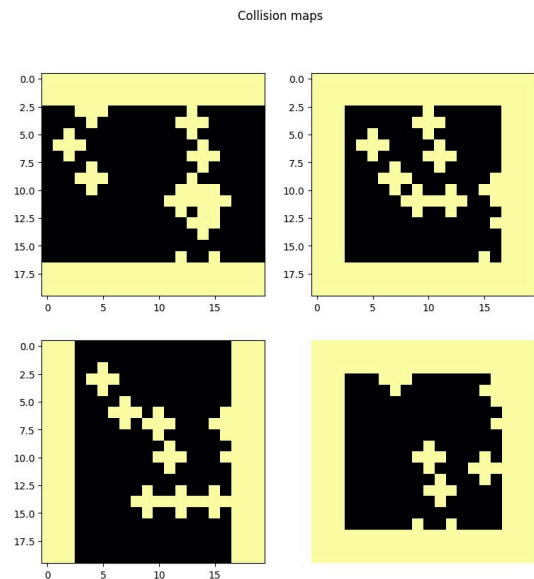


Fig. 1: Collision maps (black cells - free, bright ones - occupied)

B. Planning algorithm

In value-based RL we are finding optimal $Q^*(s, a)$, and then use greedy policy to receive optimal action in state s :

$$\pi(s_t) = \arg \max_a (Q^*(s_t, a)) \quad (1)$$

Q-learning equation:

$$Q^{(k+1)}(s_t, a_t) = Q^{(k)}(s_t, a_t) + \alpha(R + \gamma \max_{a_{t+1}} (Q^{(k)}(s_{t+1}, a) - Q^{(k)}(s_t, a_t))) \quad (2)$$

where

- $Q^{(k+1)}(s_t, a_t)$ (3)

is a new Q-value estimation

- $Q^{(k)}(s_t, a_t)$ (4)

is a former Q-value estimation

- α (5)

is the learning rate

- R (6)

is an immediate reward

- $\gamma \max_{a_{t+1}} (Q^{(k)}(s_{t+1}, a))$ (7)

is a discounted estimate of Q-value of the next stage

C. Training policies

While we don't have explicit probability transition function, we need to define policy for training Q-function. Greedy approach – to take action with maximum value, doesn't allow us to estimate other actions and we can miss optimal values. That is why, we investigate different policies:

a) *Greedy policy*:

$$\pi(s_t) = \arg \max_a (Q(s_t, a)) \quad (8)$$

b) *Softmax policy*:

$$\pi_a(s) \sim P(a|s) = \frac{\exp\left(\frac{Q(s,a)}{\tau}\right)}{\sum_{a'} \exp\left(\frac{Q(s,a')}{\tau}\right)} \quad (9)$$

c) *Upper Confidence Bound policy*:

$$A = \arg \max_a \left(Q(s, a) + c * \sqrt{\frac{\log(t)}{N(a)}} \right) \quad (10)$$

d) *Thompson sampling policy*:

$$a_t = \arg \max_a (X_a), X_a \sim N(Q(s, a), \frac{\alpha}{\sqrt{N(a)} + \beta}) \quad (11)$$

where $N(a)$ - count of choosing action a

III. EXPERIMENTS AND RESULTS

We evaluated our algorithm with such task:

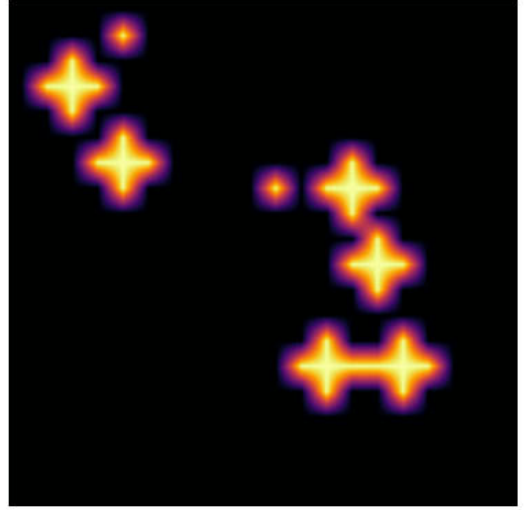


Fig. 2: Initial state (Small dots - wheels of our agent)

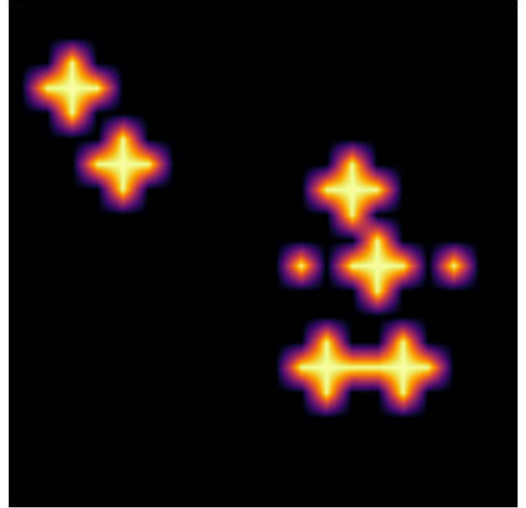


Fig. 3: Goal state

TABLE I: WITH THE NUMBER

Training episodes	Mean reward	Standart deviaton
10^3	-1.0	0.00
10^4	-1.00	0.08
10^5	0.34	0.94
$2 \cdot 10^5$	0.37	0.93
$3 \cdot 10^5$	0.09	1.00
$4 \cdot 10^5$	0.35	0.94

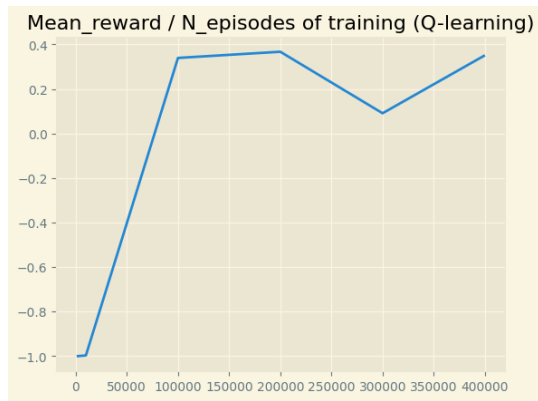


Fig. 4: Plot mean rewards/number of training episodes

From Table I and Fig. 4 we can observe that reward is growing. But we should keep in mind that received estimates are very noisy; it's connected with stochasticity of our environment and with big number of states(1600). Also we can observe that algorithm converges, after 100000 steps and further training is not necessary.

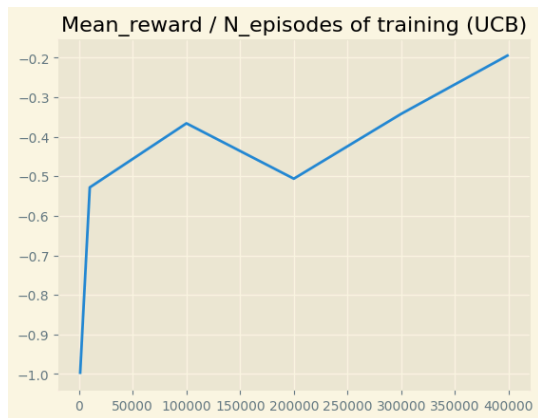


Fig. 5: Plot mean rewards/number of training episodes with ucb

With run on different policy, we get more negative values, what shows that with explore-oriented policies, our model tries different actions and get negative reward.

IV. CONCLUSIONS

In conclusion, this work proposes a method for modeling robot motion via MDP and solving it using Q-learning algorithm, allowing for dealing with environments with uncertainties and unknown dynamics. However, the discretization model used is quite crude and the approximation is weak, leading to noisy estimates. Future work could benefit from utilizing Deep RL techniques, which incorporate neural networks to improve the accuracy and efficiency of the model. This would enable more robust and reliable planning for two-wheeled robots in complex environments.

REFERENCES

- [1] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, 2018.

- [2] C. Watkins and P. Dayan, "Q-learning," vol. 8, no. 3, pp. 279–292, 1992. doi: 10.1007/BF00992698.